

Итоговый проект по дисциплине «Разработка веб-сервисов и приложений (бэкенд)»

Вам предложено на выбор 3 **индивидуальных** проекта приблизительно одинаковой сложности — разработка бэкенда различных сервисов.

Изучите их описание и выберите тот, который вас больше заинтересовал. Выбирайте вдумчиво: вы будете работать над этим проектом весь семестр.

Обратите внимание: в конце дисциплины вам предстоит ревью кода и устная защита проекта.

Таймлайн проектной работы

Дата	Этап работы
29 января	Старт работы над проектом. В личном кабинете студента открываются: • таймлайн индивидуальной проектной работы; • описание итоговых проектов; • итоговое домашнее задание, содержащее критерии оценивания проектов.
30 января — 20 мая	Проектная работа. Работа организована по принципу спринтов: 8 тем, включающих теорию и практику, — 8 этапов выполнения итогового проекта. Каждый этап завершается контрольной точкой: • 7 промежуточных контрольных точек — домашних заданий с самостоятельной проверкой; • 1 итоговая — с проверкой преподавателя. Совет: выполняйте домашнее задание до старта следующей темы (следующего вебинара), чтобы они не накапливались, и ваша нагрузка была

	равномерной. Если вы выполните все промежуточные домашние задания, ваш проект будет практически готов.
21-30 мая	Завершение работы над проектом: • доработка проекта; • деплой; • отладка; • оформление документации. В личном кабинете студента открывается регламент защиты проектов. Консультация с преподавателем: • Q&A сессия; • подведение итогов проектной работы.
30 мая	Дедлайн итогового домашнего задания. До 30 мая включительно необходимо загрузить в личный кабинет студента ссылку на репозиторий с готовым итоговым проектом.
1-14 июня	Проверка проектов преподавателем. Ревью кода.
14-18 июня	Устная защита проектов. Будет доступна запись на определённое время защиты.

Успеха в работе над проектом!

Описание проектов

1. Сервис викторин (квизов)

Описание: бэкенд сервиса для проведения квизов — быстрых командных викторин на время.

Основные функции:

- Две роли пользователей: администратор (ведущий) и игрок.
- Хранение в базе данных вопросов, ответов и результатов квизов с разделением по командам и игрокам.
- Ограничение времени ответа на вопросы с таймером: 30 секунд на ответ.

Дополнительные функции:

- Вывод статистики по игрокам и командам, подсчёт лучших игроков и команд за определённый период.

Требования к бэкенду:

1. Разработать модель данных для реализации основных функций проекта. Минимальный набор таблиц:
 - пользователи: игроки (ФИО, команда, e-mail), администраторы;
 - команды: список игроков, количество очков, количество игр/побед/поражений, ответы на вопросы;
 - вопросы: полный текст вопросов и варианты ответов с указанием правильного;
 - игры: дата и время проведения игры, участники, список вопросов и результаты игры.
2. Разработать модель данных для работы сервиса регистрации, авторизации и аутентификации пользователей. Минимальный набор таблиц:
 - пользователь: идентификатор, имя, фамилия, логин, пароль и подобные, уровень прав пользователя (администратор, игрок).
3. Разработать API для работы основных функций:
 - эндпойнты для добавления, редактирования, удаления пользователей;
 - эндпойнты для добавления, редактирования, удаления вопросов;
 - эндпойнты для добавления, редактирования, удаления игр;
 - эндпоинт для получения ответа на вопрос от команды.
4. Разработать API для регистрации, авторизации и аутентификации. Эндпойнты: регистрация, аутентификация, смена пароля, обновление токена.
5. Реализовать механизм проведения игры в реальном времени с использованием Redis Pub/Sub.
При старте игровой сессии вопросы синхронно доставляются всем

подключённым командам через подписку на уникальный Redis-канал.

Ответы от команд принимаются с обязательной проверкой на соответствие 30-секундному таймеру текущего раунда, после чего результаты фиксируются в базе данных.

Результат реализации бэкенда:

1. Разработана модель данных и настроены миграции.
2. Создан API для работы с основными функциями. Код запускается и выполняет требования задания. Допускается и приветствуется любое расширение, дополнение и обоснованное улучшение функционала.
3. Разработан набор тестов для проверки всех эндпойнтов API.
4. Развёртывание: создан файл конфигурации Docker Compose для сборки и запуска проекта в контейнере.

2. Сервис вакансий внутри компании

Описание: бэкенд сервиса для HR, позволяющего работать с вакансиями и кандидатами, а также предоставляющего HR-аналитику.

Основные функции:

- Система авторизации пользователей.
- Создание, редактирование и удаление вакансий.
- Фильтрация вакансий по статусу (открыта или закрыта) и должностям.
- Отметка закрытия вакансии.
- Создание, редактирование и удаление резюме соискателей.

Дополнительные функции:

- Подбор вакансий по параметрам соискателя (фильтрация).
- Вывод статистики по кандидатам на вакансию.

Требования к бэкенду:

1. Разработать модель данных для реализации основных функций проекта. Минимальный набор таблиц:

- вакансия: идентификатор, название, должность, описание, категория*, статус;
 - категория: идентификатор, наименование категории;
 - должность: идентификатор, наименование направления;
 - резюме: идентификатор, данные о соискателе, категория, статус.
- *Под категорией имеется в виду профессиональное направление вакансии, например, разработка, менеджмент, продажи.
2. Разработать модель данных для работы сервиса регистрации, авторизации и аутентификации пользователей: идентификатор, имя, фамилия, логин, пароль.
 3. Разработать API для регистрации, авторизации и аутентификации. Эндпойнты: регистрация, аутентификация, смена пароля, обновление токена.
 4. Разработать API для работы основных функций:
 - создание, удаление, изменение и получение вакансий;
 - создание, удаление, изменение и получение резюме;
 - фильтры по статусам и категориям вакансий;
 - фильтры по статусам и категориям резюме.

Результат реализации бэкенда:

1. Разработана модель данных и настроены миграции.
2. Создан API для работы с основными функциями. Код запускается и выполняет требования задания. Допускается и приветствуется любое расширение, дополнение и обоснованное улучшение функционала.
3. Разработан набор тестов для проверки всех эндпойнтов API.
4. Развёртывание: создан файл конфигурации Docker Compose для сборки и запуска проекта в контейнере.

3. Платформа для микрообучения

Описание: бэкенд сервиса для онлайн-обучения с возможностью покупать и проходить курсы.

Основные функции:

- Каталог курсов.
- Фильтрация и получение курсов по категориям.
- Страница курса*: бесплатная пробная версия курса (1 тема) + платный доступ к остальному содержимому. Уровни доступа для пользователей: неавторизованный пользователь (может только просматривать каталог курсов), авторизованный пользователь (может использовать пробные версии курсов), пользователь, купивший курс (может изучить все темы купленного курса).
- Система авторизации и регистрации пользователей.
- Личный кабинет пользователя с возможностями: управление курсами пользователя, хранение купленных курсов.
- Редактор курсов (для администратора).

*Страница курса — эндпоинт, по которому будет возвращена вся метainформация по курсу, доступная студенту, с разделением на платные и бесплатные части.

Дополнительные функции:

- Система статистики по курсу: количество оценок (лайков) у курса, количество обучающихся на курсе.

Требования к бэкенду:

1. Разработать модель данных для реализации основных функций проекта. Минимальный набор таблиц:
 - курс: идентификатор, название, категория, статистика, темы;
 - категория: идентификатор, наименование категории;
 - тема: идентификатор, наименование темы, содержимое (курс может содержать только текстовую информацию).
2. Разработать модель данных для работы сервиса регистрации, авторизации и аутентификации пользователей. Минимальный набор таблиц:
 - пользователь: идентификатор, имя, фамилия, логин, пароль и подобные, уровень прав (администратор, пользователь).
3. Разработать API для регистрации, авторизации и аутентификации. Эндпойнты: регистрация, аутентификация, смена пароля, обновление токена.

4. Разработать API для работы основных функций, отдельно для курсов и тем: создание, удаление, редактирование, чтение (пагинация, фильтры).
5. Разработать систему хранения оценок (лайков) и статистики популярности курсов.
6. Разработать API для покупки курсов.

Результат реализации бэкенда:

1. Разработана модель данных и настроены миграции.
2. Создан API для работы с основными функциями. Код запускается и выполняет требования задания. Допускается и приветствуется любое расширение, дополнение и обоснованное улучшение функционала.
3. Разработан набор тестов для проверки всех эндпойнтов API.
4. Развёртывание: создан файл конфигурации Docker Compose для сборки и запуска проекта в контейнере.